

Force GPS Init Removal & GPS Config Default

Date: 2026-08-19 (rolling; Supplement 2 2026-08-20) **Scope:** FC (UAVXArmQ) + GCS (uavx-python/src/) + kits + legacy emulation mirror **Status:** Implemented — builds clean (FC fc_build.py OK, GCS py_compile OK all kits)

1. Symptom reported

Pressing **Force GPS Init** in the GCS:

- USB link to the FC was **lost** (host port error), and
- the GPS module kept blinking (data still being received), but
- nothing updated on the GCS.

Diagnosis: the reset was an **independent watchdog (IWDG) trip**, not a USB fault.

2. Root cause

miscInitialiseGPS → InitGPS() ran **synchronously inside the main loop** (telem.c ProcessRxPacket → CheckTelemetry → main loop WdtFeed()). The IWDG is armed with an ~8.19 s ground reload (wdt.c:51,54, prescaler 64 @ 32 kHz LSI → 4095 steps). InitUbxGPS() blocked the main loop ≈ **9.4 s**, so WdtFeed() never ran and the watchdog reset the FC:

Step	Blocking
5 baud × 2 sends UbxInitPort (gps.c:987-990)	5 × 2 × 400 ms = 4000 ms
Delay1mS(1000) after scan (gps.c:997)	1000 ms
UbxDisableNavMessages ≈ 60 × 50 ms (gps.c:457-458,451)	~ 3000 ms
SetTimepulse/SetMode/SBAS/Interval/350ms + 2 EnableMsg	350 ms
UbxSaveConfig (gps.c:512)	1000 ms
Total	≈ 9.4 s

Boot-time InitGPS() (uavxarm-v3-gke.c:181) is immune because it runs *before* WdtInit() (:203).

3. Why removal (not a fix)

- The only viable GPS-init path on this FC is **boot-time init** or a **config-change reboot** (ParamCommit → systemReset). A live re-init therefore buys nothing a reboot doesn't already provide.
- A config change (incl. any Config bit) already forces a reboot, and the GPS bit is a **boot-time decision** — there is no live-update path by design.
- Making the init cooperative (the alternative) adds a state machine plus a protocol ACK for a feature that is redundant. **Feature removed.**

4. Changes

FC — UAVXArmQ/src/telem.c

- Enum MiscComms: slot 9 miscInitialiseGPS → **miscUnusedGPSInit** (reserved). Implicit numbering preserved so miscForceDefaults=10, miscSetNavMode=11, miscCycleB-Log=12 are unchanged on the wire.
- case miscInitialiseGPS: handler **deleted** (was: InitGPS();).

GCS — uavx-python/src/ (+ synced to linux|macos|windows/src/)

- ui/main_window.py: **Force GPS Init button removed** (layout + widget), the self.gps_init_btn.clicked.connect(self.force_gps_init) binding and the force_gps_init() method deleted. dump_black_box() restored to class-method indentation (was briefly mis-nested during edit; re-collapsed correctly).
- protocol_enums.py: GPS_INIT = 9 removed from MiscCommand; slot 9 kept as a comment (wire-stable against the FC enum).

GPS config default — GPS now on by default

- FC params.c ParamTable: pConfig2Bits default 1 (UseBatteryCompMask) → **UseBatteryCompMask | UseGPSSMask** (... | 8).
- GCS ui/parameter_window.py _load_defaults: Config2Bits default eUseFastStart → **eUseFastStart | eUseGPS**.
- Rationale: the GPS bit is the boot-time “wait for a fix before arming” gate (F.HaveGPS = UsingGPS at gps.c:1190, eInitialisingGPS at uavxarm-v3-gke.c:335). A fitted GPS should be on by default; clearing the bit remains available for airframes that must skip the fix wait.

Legacy emulation mirror — uavx-python/UAVX/

- telem.h / telem.c mirrored the same enum-slot + handler removal for consistency (not compiled into the GCS at present).

5. Verification

- FC: python3 scripts/fc_build.py → **BUILD OK** (.bin 223596 bytes).
- GCS: python3 -m py_compile OK on main_window.py, parameter_window.py, protocol_enums.py for **canonical + all three kits** (linux/macOS/windows).
- sync_kits.sh itself uses rsync (unavailable in the Flatpak sandbox), so the three changed files were mirrored with cp.
- No residual references: grep miscInitialiseGPS|GPS_INIT|gps_init|force_gps_init → clear in FC src/, GCS canonical src/, and UAVX/ mirror.

6. Notes for the user

- Flash the freshly built **UAVXArmQ/obj/FLYINGRCF4WINGMINI/FLYINGRCF4WINGMINIQ_r0.bin** to the FC.
- Existing .af files that already carry CONFIG2_BITS keep their own value (.af value wins over defaults when loaded) — the new default only affects fresh/LoadDefaults configs.
- PDF generation is the user’s job on host “Merlin” (scripts/md2pdf.sh).

Supplement 1 — Build Target Fix + GPS ISR Refactor (2026-08-19)

7. Wrong default build target (USART1 GPS never fed)

scripts/fc_build.py hardcoded the default board to **UAVXF4V3** (BOARD env default) while the Makefile defaults to **FLYINGRCF4WINGMINI** (Makefile:23). Consequences when flashing the default artifact:

- uavxf4v3.inc:138-148 maps **TelemetrySerial** = **eUsart1** and GPS to eUsart2 (only when pRxType == eCPPMRx). A GPS wired to **USART1 (PA9/PA10)** thus received bytes, but SerialISR(1) never routed them to GPSISR — the GPS LED blinks, zero GPS data in the FC/GCS.
- flyingrcf4wingmini.inc:124-130 correctly maps GPS to eUsart1 and telemetry to USB VCP.

Fix: fc_build.py default BOARD → FLYINGRCF4WINGMINI (AGENTS.md FC Build section updated). Diagnosed by the user: “default build is incorrect in the Makefile” — the Makefile was already correct; only the replica was wrong.

8. GPS packet processing moved out of the ISR

GPSISR ran ProcessGPSSentence() (validity gates, cosf, **GPSKFUpdate**, UpdateWhere) synchronously inside the USART1 interrupt. That is heavy work in ISR context and the likely cause of the **periodic ~90% exec-time bar** (exec% = 100 * execTimeuS / CurrPIDCycleuS, telem.c:559; cycle = 2 ms).

Refactor (binding principle: **ISRs are minimal execution time — only simple state machines and flag-setting; heavy math/nav runs in the main loop**):

Before	After
GPSISR: byte FSM + ParseUbxPacket() + ProcessGPSSentence() + Kalman — all interrupt context —	GPSISR / RxGPSUbxPacket: byte FSM only ; on valid checksum set F.GPSPacketReceived(gps.c:1096-1101) UpdateInertial() (inertial.c:556-567) polls the flag each PID cycle: ParseUbxPacket() → ProcessGPSSentence() → Kalman corr./UpdateWhere, then GPSKFPropagate (predict) runs in DoSensorUpdate

- gps.h: ParseUbxPacket, ProcessGPSSentence exported.
- Emulation unaffected: emu.c:647 sets F.GPSValid directly, bypassing the ISR/parse path entirely.
- Overrun note (user-acknowledged): the shared ubx.payload union is still written by the ISR; switching to a snapshot/double-buffer is a documented follow-up if inter-packet spacing ever tightens (packets » 2 ms main-loop period today, so the flag is consumed well within packet gaps).
- **Follow-up (accepted, deferred):** close the acknowledged overrun window by either (1) double-buffering the packet in the ISR (flip an index on completion — ISR stays byte-FSM only, main decodes its own buffer), or
(2) snapshot memcpy (≤384 B) of ubx.payload in the main loop at flag time before decode.
Design agreed: ISR = byte FSM + flag only; decode (ParseUbxPacket) + KF + Nav

run in `UpdateInertial`. Do not decode into GPS.* from the ISR — it recreates the same race downstream.

- `F.NewNavUpdate` (GPS fix cadence) and `F.GPSValid` now update at the PID rate from main-loop context, same 5-10 Hz data cadence, feeding `auto.c:439` as before.

9. AGENTS.md

Binding rule added under **FC Style Guide**: “ISRs must be minimal execution time — only simple state machines and flag-setting/queue-feeding; never run Kalman filters, trig/fp math, navigation, or EEPROM/flash writes from interrupt context.”

10. Verification

- FC: `python3 scripts/fc_build.py` → **BUILD OK**, `FLYINGRCF4WINGMINIQ_r0.bin` 222444 bytes (this session, at each refactor step).
- Default-board check: `fc_build.py` reports default `BOARD = FLYINGRCF4WINGMINI`.

11. Remaining investigation (user)

With the correct target flashed: “Exec time bar periodic ~90%, some acc misreads (acc cal done once), GPS LED blinking but nothing displayed.” IMU read path (`ReadIMUAccAndRate`, `icm426xx.c:84`) busy-waits on an `INT_STATUS` data-ready edge with a **5000 µs deadline** — a slow/missing `DRDY` edge could block `UpdateInertial` and inflate the exec bar; and GPS still shows nothing until the UBX frame actually parses (fail-safe to check: GPS module emitting NMEA, not UBX, after init misses — parser is UBX-only). —

Supplement 2 — GPS Pass-Through Removed + USART RX Macro Bug Found (2026-08-20)

12. GPS pass-through fully removed

The GCS “GPS Pass-Thru” feature (added 2026-08-19 to relay raw GPS bytes to the telemetry link when GPS parsing showed nothing) was **removed entirely** — FC and GCS — at the user’s request (“the pass thru code can go away and the selector marked as unused”). Raw GPS sniffing is not a viable diagnostic path: if the FC parser shows nothing, routing bytes to the host adds little without u-center, and “if we have to resort to ucenter we would be in terrible trouble” (user).

Side	File	Change
FC	<code>telem.c</code>	<code>miscGPSPassThru</code> → <code>miscUnusedGPSPassThru</code> (slot 5 kept, wire-stable); dispatch case deleted; globals <code>EnableGPSPassThru/GPSPassThruSerial</code> removed
FC	<code>sensors/gps.c</code>	<code>CheckGPSUpdate()</code> restored to plain UBX poll/parse + timeout (pass-thru bridging branch deleted)
FC	<code>telem.h</code>	extern decls for the two globals removed

Side	File	Change
GCS	protocol_enums.py	GPS_PASS_THRU = 5 → GPS_PASS_THRU_UNUSED = 5
GCS	ui/parameter_window.py	“GPS Pass-Thru” button + toggle_gps_pass_thru() removed
GCS	ui/calibration_window.py	enum in misc-name map renamed to match

Cost of removal: none. The feature was never relied on in flight; UBX parsing is unchanged. The selector byte is preserved so a stale GCS (or a recorded session) cannot accidentally re-enter a removed mode.

13. Root cause of “GPS receives nothing”: same-port USART RX pad never set to AF

Symptom (UAVXF4V3, no USB): GPS LED blinking but no GPS data; also sporadic telemetry command dropouts — consistent with dead USART **receive**.

Root cause — InitSerialPort() (boards/harness.c, all targets), GPIO block:

	Tx pin	Rx pin
Legacy (Arm32F4)	GPIO_Pin = Tx.Pin \ Rx.Pin — one	same call
Current fork (before fix)	GPIO_Init, both pads → GPIO_Mode_AF GPIO_Pin = u->Tx.Pin → AF	RX pad only re-initialised if Rx.Port != Tx.Port (harness.c:585)

For every same-port USART on the hardware (USART1 PA9/PA10, USART2 PA2/PA3, USART3 PB10/PB11, UART4 PC10/PC11), the RX pad was left in whatever the bulk init left it — GPIO_Mode_AN (analog, harness.c:857) — and GPIO_PinAFConfig() only attaches the AF *function*; an AF pad that is still in analog mode is **disconnected from the USART receiver**. RXNE never fires → no bytes in RxQ → GPS (and every same-port UART receive) dead.

This is the “USART/UART initialisation break” suspected after the USB-signalling harness rework: the cross-port if guard was added to keep the RX branch from re-touching the TX port, but it skipped the same-port case entirely.

Fix (boards/harness.c): unconditionally re-init the RX pad on its own port:

```
GPIO_StructInit(&GPIO_InitStructure);
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Pin = u->Tx.Pin;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_Init(u->Tx.Port, &GPIO_InitStructure);

/* The RX pin must be set to AF mode on its own port regardless of */
/* whether it shares the TX port ... */
GPIO_InitStructure.GPIO_Pin = u->Rx.Pin;
GPIO_Init(u->Rx.Port, &GPIO_InitStructure);
```

```
GPIO_PinAFConfig(u->Tx.Port, u->Tx.PinSource, u->USART_AF);
GPIO_PinAFConfig(u->Rx.Port, u->Rx.PinSource, u->USART_AF);
```

13.1 Blast radius (USB targets — safe)

The RX-pad init runs inside `if ((s > eUSBSerial) && (s < eSoftSerial))` (harness.c:576) — **hardware UARTs only**. `eUSBSerial` (USB VCP, `TM_USB_VCP_Init`, OTG_FS clock, PA11/12 D+/D-) and `eSoftSerial` are excluded, so **USB comms on every target are untouched**. The single shared-pad case — `flyingrcf4wingmini.inc:31 CPPMPin = PA3` (TIM2 CH4, == USART2 RX pad) — is safe by call order: `InitSerialPort()` (during `InitTarget`, harness.c:879) then `InitRC()` → `InitCPPMPin()` (harness.c:913) re-maps PA3 to TIM2 AF, last write wins. Net behaviour change everywhere: “hardware UART RX pads are correctly AF-mapped”, a strict restoration of the legacy init.

14. UAVXF4V3 GPS path verified end-to-end

- Target maps `TelemetrySerial = eUsart1` (PA9/PA10), GPS to **eUsart2** (PA2/PA3) when `pRxType == eCPPMRx` (default); CPPM capture uses **PA0** so it does not conflict with USART2 (the rc.c comment “PA3 remuxed TIM2 CH4” is a stale cross-board comment for this target; `uavxf4v3.inc:27 CPPM is PA0`).
- `InitGPS()` (`uavxarm-v3-gke.c:183`) runs unconditionally before `WdtInit()`: `SetBaudRate 115200` → `InitUbxGPS` (iNav baud-scan + UBX nav5/GNSS/interval config). **Parser is UBX-only** (`RxGPSUbxPacket`) — an NMEA-only module after a missed init produces silence; a u-blox M8/M9/M10 is required.
- **Build:** `BOARD=UAVXF4V3 python3 scripts/fc_build.py` → **BUILD OK**, `UAVXF4V3Q_r0.bin` 225396 bytes.

15. Verification

- FC grep `GPSPassThru` → only the retained enum-rename comment in `telem.c`.
 - GCS grep `GPS_PASS_THRU` → only `GPS_PASS_THRU_UNUSED` in `protocol_enums.py`.
 - `py_compile` OK: `parameter_window.py`, `calibration_window.py`, `protocol_enums.py`; FC build OK (above). (SVN commits done by the user on host; no svn binary in the Flatpak sandbox.)
-